

Layered Build Plan

AI-based detection of cyber threats in unidirectional IP traffic

Smart India Hackathon · prototype engineering plan · v1

1. Architecture review

The proposed architecture is sound. The spine — capture, parse with Zeek, extract features, run a three-branch detection engine, emit scored alerts, persist and display — is the right decomposition, and it maps cleanly onto every functional requirement in the problem statement. Choosing Zeek instead of hand-rolling packet parsing is the single best decision on the diagram; it removes weeks of work and it is what real monitoring enclaves actually run.

Five things need to change before construction starts. None of them are structural rewrites; all of them are cheaper to fix now than in week four.

1.1 Add a flow-record ingest path

The problem statement names NetFlow, IPFIX and sFlow explicitly as expected inputs. The current diagram shows only PCAP. A grader checking requirements against your architecture will mark this as a gap.

The fix is small: a second ingest adapter that consumes exported flow records and normalises them into the same internal record shape that Zeek output produces. Flow records carry no payload — only the five-tuple, byte and packet counts, and timestamps — so the DNS, TLS and packet-timing detectors cannot run on that path. State this limitation explicitly in your documentation. Knowing the boundaries of your own data reads as competence, not weakness.

1.2 Alerts must not travel through the database to reach the screen

The diagram shows a straight line from the alert stage into FastAPI and PostgreSQL, and from there into the dashboard. Read literally, that means an alert is written to Postgres and then read back out before anyone sees it. That is a batch pattern wearing streaming clothes, and it directly contradicts requirement (c), which demands incremental processing with bounded latency.

Split the path. When an alert is produced, it fans out to two consumers in parallel: a WebSocket channel that pushes it to the dashboard immediately, and a writer that persists it to PostgreSQL for history, search and evaluation. The live path never waits on the durable path. This is a ten-line change now and an awkward retrofit later.

1.3 Feature extraction is not one box

The hardest engineering in this entire system is hiding inside the words "feature extraction". Turning one Zeek log line into a few numeric fields is trivial. What the detectors actually need is *stateful windowed aggregation* at several granularities at once:

- Per source IP over short windows — feeds the flood and scan detectors
- Per source-destination-port tuple over long windows — feeds beacon detection
- Per internal host, as a slowly-updating baseline — feeds exfiltration and fingerprint-rarity scoring
- Per parent domain — feeds DNS tunnelling

All of that has to run in bounded memory while traffic streams through. Give it its own named layer with its own owner. Teams that treat it as a helper function discover in week three that it is actually the project.

1.4 Insert a fusion layer between detection and alerting

Three detection branches producing raw scores cannot emit alerts directly. Three problems appear immediately:

- A rule outputs a boolean, a classifier outputs a probability, an anomaly detector outputs an unbounded distance. These are not comparable, and the problem statement requires a confidence score that means something.
- A beacon that fires every sixty seconds for an hour generates sixty identical alerts. That is an unusable dashboard.
- A host that beacons, then opens an unusual encrypted session, then uploads a large volume, is one incident. Emitting it as three unrelated alerts throws away the most valuable signal you have.

The fusion layer calibrates scores onto a common scale, suppresses duplicates within a time window, and correlates related alerts into incidents. This is the layer that separates a working prototype from a pile of scripts, and almost no competing team will build it.

1.5 Scope the AI analyst layer deliberately

The RAG analyst at the bottom of the diagram is a real differentiator, and it plays to the team's existing strengths. It is also the layer most likely to consume time that the six detectors needed.

Keep it strictly downstream and strictly read-only against the alert store. It should explain an alert in plain language, narrate a correlated incident, and answer questions like "what happened on this host in the last hour" by querying persisted alerts. It should never influence detection, never change a score, and never be on the critical path for producing an alert. If the analyst breaks or is switched off, the system must still fully satisfy the problem statement.

ANTICIPATE THIS QUESTION

A judge will ask whether the LLM has network access, and whether that violates the read-only constraint. Have the answer ready. Calling a hosted model API is egress from the monitoring enclave, which is defensible but invites argument. Running a local model removes the question entirely and is the stronger demo. If you must use a hosted API, document the boundary explicitly and show that the ingest path remains one-way regardless.

1.6 Corrected architecture



Everything below builds this, bottom-up, one layer at a time.

2. Layer-by-layer build

Each layer below states its purpose, what to build, the decisions to make once and not revisit, and a concrete completion test. Build them in order. Do not start a layer until the one below it passes its test.

LAYER 0

Contracts and scaffolding

Purpose

Unblock parallel work. Until the data shapes are fixed, every person on the team is waiting on someone else. This is the highest-leverage two hours of the entire project.

What to build

Two frozen JSON schemas, a repository skeleton, and a fixture file of synthetic records that conform to them.

The flow record — what leaves the parsing layer and enters the feature store:

```
{
  "ts": 1735689600.123,
  "uid": "CHhAvVGS1DHFjwGM9",
  "src_ip": "10.0.0.15", "src_port": 51422,
  "dst_ip": "93.184.216.34", "dst_port": 443,
  "proto": "tcp",
  "duration": 2.41,
  "orig_bytes": 1204, "resp_bytes": 88213,
  "orig_pkts": 14, "resp_pkts": 62,
  "conn_state": "SF",
  "dns": { "query": "...", "qtype": "A", "rcode": "NOERROR" },
  "tls": { "sni": "...", "ja3": "...", "ja3s": "...", "version": "TLSv13" }
}
```

The `dns` and `tls` objects are present only when the protocol applies, and never present on the flow-record ingest path.

The alert record — the system's actual output, and the thing the problem statement grades:

```
{
  "alert_id": "a7f3...",
  "ts": 1735689612.880,
  "flow_id": "CHhAvVGS1DHFjwGM9",
  "src_ip": "10.0.0.15", "dst_ip": "93.184.216.34",
  "threat_class": "c2_beaconing",
  "confidence": 0.87,
  "severity": "high",
  "detector": "beacon_periodicity",
  "detector_version": "0.3.1",
  "evidence": [
    { "feature": "interval_cv", "value": 0.04, "threshold": 0.15 },
    { "feature": "observed_periods", "value": 42 },
    { "feature": "mean_interval_s", "value": 60.2 }
  ],
  "mitre_technique": "T1071.001",
  "incident_id": "i2c9..."
}
```

Decisions to freeze now

- Field names and types in both schemas. Changing these in week three costs a day of everyone's time.
- The threat class identifiers, as fixed strings. Note that the six classes in the problem statement map to eight detectors, since volumetric flooding and amplification behave differently, as do DGA and DNS tunnelling.
- Severity vocabulary: `low`, `medium`, `high`, `critical`.
- Time is always a Unix float in UTC. No local time anywhere, ever.
- Every alert carries evidence. A detector that cannot say *why* it fired does not ship.

DONE WHEN

Both schemas are committed with a validator. A fixture file of a few hundred synthetic flow records and a few dozen synthetic alerts exists in the repository. The dashboard developer can build a full UI against those fixtures without any other layer existing.

LAYER 1

Ingest

Purpose

Get traffic into the system at a controllable, repeatable rate, from both input types the problem statement names.

What to build

Packet path. A replay harness that pushes a capture file onto a network interface at a configurable rate, plus a live-capture mode reading directly from an interface. Rate control is not optional — it is the mechanism by which you will later demonstrate your throughput figure. Build the speed multiplier and the packets-per-second cap on day one.

Flow path. A collector that receives exported flow records over UDP and an adapter that normalises them into the Layer 0 flow record shape, leaving the protocol-specific objects absent.

Decisions to freeze now

- Run the whole lab on Linux, in containers, on the always-on machine. Packet capture, raw sockets and most traffic generators are Linux-native and fight you elsewhere.
- The container bridge interface is your span port. Everything is captured there.
- Replay is always deterministic and repeatable. The same input file must produce the same alerts. Without this you cannot debug anything.

DONE WHEN

You can replay a capture file at 1x, 5x and 20x, and observe packet counts on the capture interface that match expectations. A flow record sent to the collector appears as a normalised record on the other side.

LAYER 2

Parsing

Purpose

Turn packets into structured, protocol-aware conversation records.

What to build

Zeek, configured to write its logs as newline-delimited JSON, with the JA3 fingerprinting plugin enabled. Then a thin reader that tails the log files and emits Layer 0 flow records.

The logs that matter:

Log	Feeds
conn.log	Every detector. Flow tuple, byte counts per direction, duration, connection state.
dns.log	DGA and DNS tunnelling. Query name, record type, response code.
ssl.log	Encrypted-session detection. TLS version, SNI, JA3, JA3S.
x509.log	Certificate features: validity window, self-signed status.

Decisions to freeze now

- Do not write your own packet parser. This is not a shortcut worth taking, and Zeek is more defensible in front of judges than anything you could write in six weeks.
- Join `conn.log` to the protocol logs on Zeek's connection UID, not on the five-tuple. The UID is exactly what it exists for.
- Late-arriving protocol data is normal. A DNS or TLS record may land after the connection record. Handle it, do not fight it.

DONE WHEN

Replaying a capture produces Layer 0 flow records with DNS and TLS fields correctly populated, validated against the Layer 0 schema. JA3 hashes are present on TLS connections.

LAYER 3

Feature store

Purpose

Maintain the rolling state that every detector reads from. This is the technical heart of the system.

What to build

A stateful aggregator keeping several window families in memory simultaneously:

Key	Window	What it accumulates
source IP	5s, 60s, 5min	packet and byte rate, distinct destination IPs, distinct destination ports, connection-state distribution
destination IP	5s, 60s	distinct sources, source-IP entropy, SYN to SYN-ACK ratio
src, dst, dport	30min – 6h	connection timestamps, inter-arrival series, byte-size consistency
parent domain	5min, 1h	distinct subdomains, mean query length, record-type mix, NXDOMAIN rate
internal host	rolling baseline	typical outbound volume, known destinations, seen JA3 fingerprints, active hours

Decisions to freeze now

- **Bound the memory.** A naive set of destination ports per source IP dies the moment a spoofed flood arrives. Use probabilistic structures: HyperLogLog for distinct counts, Count-Min Sketch for heavy hitters, a streaming estimator for entropy. This is a genuine engineering differentiator and worth a slide.
- Expire state on a timer, not on memory pressure. Predictable eviction beats clever eviction.
- The store exposes one read interface. Detectors never touch raw logs.
- Two operating modes: *learning*, which updates baselines and emits nothing, and *detection*, which reads baselines and scores. The mode switch is explicit and visible.

DONE WHEN

Replaying a capture produces correct window aggregates that you have verified by hand against a small sample. Memory usage stays flat across a long replay containing a high-cardinality flood.

LAYER 4

Detection engine

Purpose

Read features, produce scored findings. Three branches, one plug-in interface.

The interface

Every detector, regardless of branch, implements the same contract: it receives a feature view, and returns either nothing or a finding carrying a raw score and the evidence features that produced it. Nothing else. Detectors do not write to the database, do not talk to each other, and do not decide severity.

Branch A — rules and statistics

Build these first. Half of the eight detectors need no machine learning at all, they run in microseconds, and they are trivially explainable when a judge asks why an alert fired.

Detector	Core signal
Port scanning	Distinct destination ports or hosts per source per window; ratio of attempts to established connections. Separate vertical, horizontal and block patterns.
Volumetric DDoS	Source-IP entropy against packet rate. High entropy with no completed handshakes indicates spoofing; low entropy with high volume indicates a direct flood.
Amplification	Response-to-request byte ratio on UDP toward known reflector ports.
DNS tunnelling	Subdomain cardinality under one parent, mean query length, unusual record-type mix, sustained query rate to one authority.

Branch B — machine learning

Two of the eight genuinely benefit from a trained model. Spend your modelling effort here and nowhere else.

Detector	Approach
DGA classification	Character-level model over the queried label: entropy, n-gram frequency against a legitimate-domain corpus, length, digit ratio, consonant runs. Combine with the behavioural signal of NXDOMAIN bursts from one host, which is often stronger than the string itself.
Encrypted-session malware	Model over TLS metadata: JA3 rarity against the learned baseline, absence of SNI, certificate validity window, self-signed status, and the packet-size and inter-arrival sequence of the first several packets. Never any payload.

Branch C — anomaly detection

Unsupervised scoring against per-host baselines, for behaviour you cannot enumerate in advance.

Detector	Approach
Data exfiltration	Outbound volume against the host's own learned baseline, outbound-to-inbound ratio, destination novelty, transfer duration, time-of-day deviation.
C2 beaconing	Regularity of the inter-arrival series: coefficient of variation, autocorrelation, spectral peak. Must tolerate jitter, since real beacons randomise their interval.

THE TWO THAT WILL BE ATTACKED IN QUESTIONING

Exfiltration and beaconing are where false positives live. Legitimate backups and video calls look exactly like exfiltration on a byte-ratio anomaly. Time synchronisation, package update checks and monitoring heartbeats are perfectly periodic and entirely innocent. If your benign traffic does not contain these confounders, your numbers are meaningless and a knowledgeable judge will say so. Plant them deliberately — see section 3.

DONE WHEN

All eight detectors run against a replayed capture and produce findings with populated evidence. Each is independently unit-tested against fixture features with known expected outputs.

LAYER 5

Fusion

Purpose

Convert incomparable raw findings into calibrated, deduplicated, correlated alerts. This layer is what makes the output usable, and it is what most competing teams will not have.

What to build

Calibration. Map each detector's raw score onto a common probability scale by fitting against your labelled validation set. A confidence of 0.8 must mean the same thing whichever detector produced it, otherwise the number on the dashboard is decorative.

Severity assignment. Severity is not confidence. It combines confidence with the inherent seriousness of the threat class and, if you model it, the importance of the asset involved. A low-confidence exfiltration finding on a critical host may outrank a high-confidence scan on a spare machine.

Deduplication. Suppress repeat findings for the same entity, threat class and evidence pattern within a time window. Emit one alert with an occurrence count rather than sixty identical alerts.

Correlation. Group related alerts on the same host within a time window into a single incident, and order them into a kill-chain narrative: reconnaissance, then command and control, then exfiltration. This is the highest-value component in the entire build relative to its cost.

DONE WHEN

A one-hour replay containing a persistent beacon produces one incident with an occurrence count, not sixty alerts. Confidence scores from different detectors are demonstrably comparable against held-out labelled data.

LAYER 6

Alert bus and persistence

Purpose

Deliver alerts to the screen immediately and to storage durably, without either path blocking the other.

What to build

A publisher that fans each alert out to two independent consumers. One pushes over a WebSocket to connected dashboards. The other writes to PostgreSQL. The live path must never wait on the durable path.

Schema: an `alerts` table with the evidence array stored as JSONB, an `incidents` table, and a join between them. Index on timestamp, source IP, and threat class — those are the three things every query filters on.

Decisions to freeze now

- Batch the database writes. One insert per alert will become your bottleneck during a flood, which is precisely when you least want a bottleneck.
- Stamp every alert with the timestamp of the packet that triggered it and the timestamp of alert emission. The difference is your detection latency, and you are required to report it.
- The API is read-only over historical data. Nothing in this system ever writes back toward the network.

DONE WHEN

Alerts appear on a connected WebSocket client within a bounded, measured delay of the triggering packet, and are simultaneously queryable from PostgreSQL by time range, host and threat class.

LAYER 7

Dashboard

Purpose

This is what the judges actually see. Every other layer is evaluated through this one.

What to build

- **Live alert feed** — streaming in via WebSocket, filterable by threat class, severity and host.
- **Evidence drill-down** — click any alert, see the exact features and values that triggered it against their thresholds. This directly answers the "supporting evidence" requirement and is the most convincing thing on the screen.
- **Incident view** — correlated alerts on one host shown as an ordered narrative.
- **Throughput panel** — live flows per second, packets per second, and megabits per second sustained. Requirement (d) asks you to demonstrate this, not assert it.
- **Latency panel** — distribution of packet-to-alert delay, showing median and tail.
- **Host timeline** — one machine's activity over time, which is how an analyst actually investigates.

BUILD THIS AGAINST FIXTURES FROM DAY ONE

The dashboard developer should never be blocked. Layer 0 fixtures let the entire interface be built and polished while the pipeline is still being written. Swapping the fixture source for the real WebSocket at the end is a one-line change.

DONE WHEN

A live replay drives the full dashboard, and any alert can be drilled into for its evidence within two clicks.

LAYER 8

AI analyst

Purpose

Differentiation, not detection. Strictly downstream, strictly optional to the core requirements.

What to build

- Plain-language explanation of any alert, generated from its evidence fields.
- Incident narration — turn a correlated group into a short account of what appears to have happened.
- Natural-language query over the persisted alert store: "show me every host that beaconed in the last hour".
- Suggested analyst next steps, framed as observations rather than instructions.

Hard boundaries

- Reads only from the alert database. Never from raw traffic, never from payload.
- Cannot influence detection, scoring or severity.

- Every generated statement traces back to specific evidence fields. An analyst layer that speculates beyond its evidence is worse than no analyst layer.
- The system passes every problem-statement requirement with this layer switched off. Demonstrate that if asked.

DONE WHEN

Any alert or incident can be explained in plain language, with every claim traceable to a stored evidence field, and the whole layer can be disabled without affecting anything upstream.

LAYER 9

Evaluation and benchmark harness

Purpose

Turn claims into measurements. This layer is a graded deliverable disguised as internal tooling.

What to build

Accuracy harness. Replay a labelled capture, join emitted alerts against ground truth, and report precision, recall and F1 *per threat class*. Report false positives per hour on purely benign traffic as a headline number.

DO NOT REPORT ACCURACY

With rare attack classes, overall accuracy is a meaningless figure — a detector that fires on nothing scores extremely well. Any judge who knows the domain will notice immediately. Report precision and recall per class, and be ready to explain the trade-off you chose.

Throughput harness. Replay at increasing rates until the pipeline saturates. Record sustained flows per second, megabits per second, per-layer CPU and memory, and the latency distribution at each rate. The output is one chart and one sentence: "*sustained X flows per second with 95th-percentile alert latency under Y milliseconds.*" That sentence is what requirement (d) is asking for.

Constraint proof. A demonstration that runs the entire system with outbound network access blocked, showing it still detects normally. Thirty seconds of demo for the one architectural constraint every other team will treat as decorative.

DONE WHEN

A single command produces the full evaluation report, and the throughput figure is a number you measured rather than one you estimated.

3. Parallel workstream: traffic generation

The problem statement provides no dataset link. What it provides is a recipe. This means building the data is a workstream in its own right, and it starts on day one and runs alongside everything above.

The upside is real: traffic you generate yourself carries perfect ground-truth labels, because you know exactly which container sent which packet at which second.

3.1 The gap in the named tools

Threat class	Coverage from the named tools
Volumetric DDoS	Covered by the named flood and slow-exhaustion tools
Amplification	Needs a reflector service standing in the lab
DNS tunnelling	Covered by the named tunnelling tools
DGA	Partially — published sources give domain <i>strings</i> , not traffic. You must resolve them from a lab host to produce the DNS flows.
C2 beaconing	Underspecified. Write your own emulator.
Port scanning	Not named. Trivial to add.
Data exfiltration	Not named. A scripted bulk upload covers it.
Encrypted-session malware	Not named. This is the real gap.

Working around the encrypted-traffic gap honestly. Have the beacon emulator connect over TLS using a non-browser HTTP client. Scripting-language HTTP libraries and command-line tools each produce a JA3 fingerprint distinctly unlike a mainstream browser, because they offer different cipher suites in a different order. That is precisely the real-world signal analysts use, and it is fully defensible in your documentation. Do not imply you have malware captures you do not have; state the substitution and explain why it is valid.

Write your own beacon emulator. A small script that contacts a listener every N seconds with configurable jitter and payload size gives you deterministic, parameterisable, perfectly labelled traffic. More importantly, it lets you sweep the jitter parameter and find the exact point where your detector fails. "Our beacon detector holds to 40% jitter and degrades beyond that" is a far stronger claim than "we detected the beacon".

3.2 The trap that decides your credibility

Load generators produce bulk streams and packet floods. Neither resembles real background traffic. If benign traffic comes from a load generator and malicious traffic comes from an attack tool, your models will hit near-perfect scores within days — but they will have learned to distinguish *one tool from another tool*, which has nothing to do with security. Worse, your beacon detector will report zero false positives, because nothing in your benign set is legitimately periodic.

You will have never tested the hard part, and you will not find out until a judge asks.

Plant confounders deliberately:

Detector under test	Legitimate traffic that resembles the threat
Beaconing	Time synchronisation, package update checks, monitoring agent heartbeats, telemetry — all perfectly periodic and entirely innocent
Exfiltration	Cloud backup, video call, large file upload — all high outbound-to-inbound ratio
DGA	Content-delivery hostnames and cloud storage buckets — long, random-looking, legitimate
Scanning	Asset inventory or vulnerability sweeps — authorised recon, structurally identical to hostile recon

Source some of this by letting a few lab containers reach the real internet: a headless browser session, a package update, a time daemon. Real traffic beats synthetic traffic for the benign half in every case.

This is your differentiator. Most teams will report near-perfect scores on a clean split. Being able to say "our benign set contains four legitimate periodic services and our beacon detector separates them at 94% precision" puts you in a

different conversation entirely.

3.3 Label by construction

Never label after the fact. Build a scenario runner that orchestrates each attack and writes a manifest as it executes: start time, end time, source, destination, threat class. Assign attacker containers to a reserved address range so anything from that range inside that window is unambiguous.

A labeller then joins the manifest against the connection log on five-tuple and timestamp, and you have labelled flows for free. The whole thing is on the order of 150 lines, and it is what makes every number in your evaluation real.

Keep the manifest in version control. When a judge asks how you validated, you open the scenario file.

3.4 Known limitation, stated up front

Containers on a shared bridge have near-zero round-trip time, no jitter, and related hardware addresses. Some of your timing features will look unnaturally crisp compared to real network conditions. Let the benign side reach the real internet where you can, introduce artificial delay and loss on the lab bridge, and document the limitation in your writeup. Naming a weakness before a judge finds it reads as rigour.

4. Team allocation

Role	People	Owns
Pipeline	2	Layers 1-3. Ingest, Zeek integration, feature store, windowing, memory bounds. The hardest infrastructure work in the project.
Detection	2	Layer 4 and Layer 5. One person takes the rule and statistical detectors plus fusion; the other takes the two ML detectors and calibration.
Interface	1	Layers 6-8. Alert bus, API, dashboard, AI analyst. Starts immediately against Layer 0 fixtures and is never blocked.
Data and evaluation	1	Section 3 and Layer 9. Traffic generation, labelling, benchmark harness, and the required documentation.

The last role looks like the least glamorous seat on the team. It is the one that determines whether anything you build can be proven on demo day. Give it to someone rigorous, not to whoever is left over.

5. Build sequence

Milestone	Target
M1	Layer 0 complete. Schemas frozen, fixtures committed, everyone unblocked.
M2	Thin slice. One capture file produces one port-scan alert on the dashboard, end to end. Ugly code and hardcoded thresholds are acceptable. This proves the pipe exists.
M3	Traffic generation working. Labelled captures for all eight detectors, including planted benign confounders.
M4	All four rule and statistical detectors passing on labelled data. Feature store memory-bounded and verified.
M5	Both ML detectors trained and evaluated. Fusion layer calibrating, deduplicating and correlating.
M6	Benchmark harness producing throughput, latency and per-class accuracy figures.
M7	AI analyst layer, dashboard polish, documentation, demo rehearsal.

M2 is the milestone that matters psychologically. Once a single alert has travelled from a capture file to a screen, the project stops being intimidating and becomes a matter of filling in boxes.

6. Risk register

Risk	Severity	Mitigation
Benign traffic too clean; models learn tool artefacts rather than threat behaviour	Critical	Plant confounders per section 3.2. Include real internet traffic in the benign set.
Feature store underestimated; becomes the bottleneck in week four	High	Give it a named owner and its own layer. Use probabilistic structures from the start, not as a later optimisation.
Exfiltration detector generates unusable false-positive volume	High	Per-host baselining plus destination novelty. Tune for precision over recall and say so explicitly.
AI analyst absorbs time the detectors needed	Medium	Hard-gate it behind M6. The system must fully pass the problem statement without it.
No throughput figure by demo day	Medium	Rate control built into Layer 1 on day one, not retrofitted at the end.
Architecture churn	Medium	Layer 0 schemas are frozen. Changes require the whole team to agree, not one person to decide.

7. Demo-day checklist

Six things must be ready to show, in this order of importance:

1. **Live detection.** Replay running, alerts appearing on the dashboard in real time.
2. **Evidence drill-down.** Click an alert, show exactly which features and values triggered it. This answers the requirement most teams gesture at.
3. **An incident, not an alert list.** One host, correlated activity, ordered into a narrative.
4. **The throughput number.** Live gauge plus a sentence: sustained rate and tail latency, both measured.
5. **Per-class precision and recall** against labelled data, with an honest statement of where the system is weakest.
6. **The read-only proof.** System running with outbound access blocked, still detecting. Thirty seconds, and almost nobody else will do it.

Have an answer prepared for each of these: how you controlled false positives, why you chose statistical methods where you did, what your benign baseline contains, where the system fails, and whether the AI layer has network access.